

Vision Document
Master Thesis Proposal

**Title: Extending Jupyter Notebook
Reproducibility Analysis to Non-GitHub
Repositories (Codeberg, Zenodo)**

Supervisor: Dr. Sheeba Samuel

Student: David Keci

Program: M.Sc. Web Engineering

University: TU Chemnitz

Date: March 2026

1. Introduction

Jupyter notebooks are widely used to share computational workflows by combining code, text, and results in a single document. They play an important role in reproducible research, but in practice many notebooks cannot be reliably executed in new environments.

Existing work has mainly focused on executing notebooks and measuring reproducibility rates. These studies show that failures are common, often caused by missing dependencies, incomplete environment specifications, or hidden execution states [1], [2]. However, most of this research is limited to notebooks hosted on GitHub, while other platforms such as Codeberg and Zenodo are largely ignored.

At the same time, tools like Repo2Docker and Papermill support automated execution, but depend on the availability of correct metadata [3], [4]. The FAIR Jupyter approach further improves accessibility by structuring reproducibility data into a knowledge graph, yet it remains limited to GitHub-based datasets and does not address cross-platform differences [5].

This thesis extends existing work by introducing a cross-platform pipeline for collecting and analyzing notebooks from GitHub, Codeberg, and Zenodo. It focuses on standardizing metadata across platforms and enabling comparative reproducibility analysis. In addition, the approach explores lightweight automated support to improve metadata completeness before execution, aiming to reduce avoidable failures.

2. Problem Statement

Even though many studies have analyzed reproducibility in Jupyter notebooks, there are still some important problems that are not solved.

First, most research only looks at notebooks from GitHub. This is a problem because notebooks also exist on other platforms like Codeberg and Zenodo. These platforms are different in how they store data and metadata. If we only study GitHub, we do not get the full picture of how reproducibility works across all platforms.

Second, reproducibility depends heavily on metadata, such as dependencies, environment setup, and configuration. In many real notebooks, this information is missing, incomplete, or inconsistent. Current tools expect this metadata to already exist, and when it does not, the notebook fails. There is also no common format to make metadata consistent across different platforms, which makes comparison difficult.

Third, most existing approaches only check reproducibility after running the notebook. They execute the notebook and then report whether it worked or failed. However, they do not consider differences between types of notebooks before execution. Notebooks can vary significantly, for example between machine learning experiments, data analysis tasks, or simple tutorials, and these differences can influence reproducibility outcomes. Current approaches generally treat notebooks

in a uniform manner and do not explicitly classify them into different types. As a result, they do not systematically analyze how different categories of notebooks may influence reproducibility outcomes.

Because of these issues, current research is limited. It does not consider multiple platforms together, it does not handle inconsistent metadata well, and it does not improve the conditions before execution.

So, the main problem of this thesis is: *How can we analyze reproducibility of Jupyter notebooks across different platforms while making sure the metadata is consistent and complete enough for reliable execution?*

3. State of the Art

Existing research on Jupyter notebook reproducibility can be grouped into four main directions.

Execution-based approaches run notebooks to check whether they execute successfully. They attempt to reconstruct environments using files such as requirements.txt or environment.yml and then execute all cells. While effective for measuring success or failure, they mainly report outcomes and error logs, depending on incomplete metadata, and are typically limited to notebooks from GitHub [1], [2], [3], [4].

Knowledge graph-based approaches organize notebook metadata into structured representations. They model relationships between notebooks, dependencies, and other elements to enable better querying and analysis. However, they do not execute notebooks and therefore cannot directly verify reproducibility. These approaches are also usually limited to a single platform [5].

Machine learning and natural language processing approaches analyze notebook content to detect patterns related to reproducibility. They can be used to identify issues such as missing dependencies, predict execution failures, and in some cases classify notebooks or their characteristics. These approaches are typically developed as separate components [6].

Empirical software engineering studies analyze large collections of notebooks to identify common reproducibility issues such as missing dependencies or incorrect execution order. While they provide useful insights, they do not offer integrated solutions [7].

Overall, existing approaches address specific aspects of reproducibility but are not combined into a unified system. In addition, most work focuses on GitHub and does not consider cross-platform differences across environments such as Codeberg and Zenodo.

4. Research Objectives

- Develop a cross-platform pipeline to collect Jupyter notebooks from GitHub, Codeberg, and Zenodo.
- Design a metadata normalization approach to standardize dependencies, environments, and execution information across platforms.
- Develop a knowledge graph representation for cross-platform notebook metadata and reproducibility information .
- Compare execution success rates and metadata quality of notebooks across GitHub, Codeberg, and Zenodo.
- Investigate the use of automated methods to classify notebooks and analyze how notebook type influences reproducibility

5. Proposed Approach

This thesis proposes a cross-platform pipeline for analyzing the reproducibility of Jupyter notebooks, with a primary focus on metadata standardization and comparative analysis across platforms. The approach builds on the FAIR Jupyter pipeline by adapting its data model and extending it to support additional platforms such as Codeberg and Zenodo.

Cross-Platform Notebook Collection

Notebooks are collected from multiple platforms, including GitHub, Codeberg, and Zenodo. Platform-specific connectors are implemented to retrieve notebooks and their associated metadata in a consistent format, despite differences in APIs and storage structures.

Metadata Processing and Normalization

Metadata such as dependencies, environment configurations, and execution-related information is extracted from available sources, including configuration files and notebook content. This metadata is then transformed into a unified schema to ensure consistency across platforms.

In cases where metadata is incomplete, additional information is derived directly from the notebook content, for example by identifying dependencies from import statements. This step improves metadata quality and reduces avoidable execution failures caused by missing or inconsistent information.

Knowledge Graph Representation

The normalized metadata is represented using a knowledge graph model that builds on the FAIR Jupyter data model. The graph includes entities such as notebooks, dependencies, and platforms, as well as the relationships between them. Platform provenance is explicitly represented, enabling notebooks from different sources to be distinguished and analyzed within a unified structure.

Execution and Data Collection

Execution results are obtained from available datasets and prior studies where possible. These results, such as success or failure, are integrated and linked with the corresponding metadata. For notebooks without existing results, execution can be performed selectively using tools such as Papermill.

Notebook Classification

Notebooks are categorized into different types based on their content and usage patterns using automated methods where applicable. This classification supports the analysis of how different categories of notebooks may exhibit different reproducibility patterns.

Comparative Analysis

The final step compares execution success rates and metadata quality across platforms. This analysis is used to identify differences between GitHub, Codeberg, and Zenodo, and to evaluate how platform-specific characteristics influence reproducibility outcomes.

6. Evaluation Strategy

To evaluate the proposed approach, both quantitative and qualitative analyses will be conducted. The evaluation focuses on assessing the effectiveness of the cross-platform pipeline, the quality of metadata normalization, and the impact on reproducibility outcomes.

First, the pipeline will be evaluated based on its ability to successfully collect and process notebooks from multiple platforms, including GitHub, Codeberg, and Zenodo. Metrics such as the number of retrieved notebooks, completeness of extracted metadata, and success rate of metadata normalization will be measured.

Second, reproducibility will be evaluated by analyzing execution outcomes. The proportion of successfully executed notebooks will be compared before and after metadata normalization to assess whether the proposed approach improves execution success rates. Failure cases will also be analyzed to identify common causes and determine whether metadata enrichment reduces avoidable errors.

Third, a comparative analysis across platforms will be conducted. This includes comparing execution success rates, metadata completeness, and types of failures between GitHub, Codeberg, and Zenodo. The goal is to determine whether platform-specific characteristics influence reproducibility outcomes.

Finally, the effect of notebook classification will be evaluated by analyzing reproducibility patterns across different notebook types. This will help assess whether certain categories of notebooks are more prone to reproducibility issues and whether classification provides meaningful insights.

7. Conclusion

This thesis addresses the limitations of current reproducibility research on Jupyter notebooks by moving beyond a GitHub-centered perspective and introducing a cross-platform approach. By extending the FAIR Jupyter pipeline to support Codeberg and Zenodo, the work enables a more comprehensive and less biased analysis of reproducibility across different repository ecosystems.

The expected outcome is a production-ready extension of the existing pipeline, along with a cross-platform dataset and comparative analysis. This will contribute to a deeper understanding of how platform characteristics and metadata quality affect reproducibility and provide a foundation for future research and further extensions to additional platforms.

References

- [1] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks,” in *Proc. IEEE/ACM 16th Int. Conf. on Mining Software Repositories (MSR)*, Montreal, Canada, 2019, pp. 507–517, doi: 10.1109/MSR.2019.00077
- [2] S. Samuel and D. Mietchen, “Computational reproducibility of Jupyter notebooks from biomedical publications,” *GigaScience*, vol. 13, pp. 1–23, 2024, doi: 10.1093/gigascience/giad113
- [3] Project Jupyter, “Repo2Docker,” GitHub repository. [Online]. Available: <https://github.com/jupyterhub/repo2docker>
- [4] nteract, “Papermill,” GitHub repository. [Online]. Available: <https://github.com/nteract/papermill>
- [5] S. Samuel et al., “FAIR Jupyter: A Knowledge Graph-Based Approach to Reproducible Research,” *Trans. Graph Data Knowl. Discov.*, vol. 2, no. 2, 2024, doi: 10.4230/TGDK.2.2.4
- [6] S. Samuel and D. Mietchen, “Computational reproducibility of Jupyter notebooks from biomedical publications,” arXiv:2209.04308, 2022
- [7] “Reproducibility in Jupyter Notebooks: An Empirical Study,” in *Proc. IEEE/ACM MSR*, 2025, doi: 10.1109/MSR66628.2025.00070